**A MINI PROJECT REPORT ON**

# AI-GENERATED IMAGE DETECTION USING DEEP LEARNING

*Submitted in partial fulfillment of the requirements for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**

*By*

**BOJJA SAHASRA (Roll No: 2401100CSE0243)**

**MUDIGONDA LALITHA SREYA (Roll No: 2401100CSE0264)**

**ROHAN JOSHUA FRANCIS (Roll No: 2401100CSE0275)**

*Under the Esteemed Guidance of*

**Dr. SARITHA BANTU**

**Professor, Department of Computer Science and Engineering**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**SREENIDHI UNIVERSITY**

**HYDERABAD, TELANGANA, INDIA**

**ACADEMIC YEAR: 2026–2027**

# SREENIDHI UNIVERSITY

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## CERTIFICATE

This is to certify that the mini-project entitled "AI-Generated Image Detection Using Deep Learning" is a bonafide record of work carried out by Bojja Sahasra (Roll No: 2401100CSE0243), Mudigonda Lalitha Sreya (Roll No: 2401100CSE0264), and Rohan Joshua Francis (Roll No: 2401100CSE0275) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Sreenidhi University during the academic year 2026–2027.

The results embodied in this report have been verified and have not been submitted to any other University or Institution for the award of any degree or diploma.

**Internal Guide**

**Dr. Saritha Bantu**
**Professor, Dept. of CSE**
**Sreenidhi University**

**Head of the Department**

**Department of CSE**
**Sreenidhi University**

# DECLARATION

We hereby declare that the mini-project report entitled "AI-Generated Image Detection Using Deep Learning" submitted to the Department of Computer Science and Engineering, Sreenidhi University, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is an authentic record of our original work conducted under the guidance of Dr. Saritha Bantu, Professor, Department of CSE.

We further declare that the content of this project has not formed the basis for the award of any degree, diploma, associateship, or other similar title in this or any other university or higher education institution.

Date: 20-10-2026

Place: Hyderabad, Telangana

Team Members:

1. Bojja Sahasra (2401100CSE0243) _____
2. Mudigonda Lalitha Sreya (2401100CSE0264) _____
3. Rohan Joshua Francis (2401100CSE0275) _____

# ACKNOWLEDGEMENT

We express our sincere gratitude and indebtedness to our esteemed faculty guide, Dr. Saritha Bantu, Professor, Department of Computer Science and Engineering, Sreenidhi University, for her valuable guidance, constructive suggestions, and continuous encouragement throughout the conceptualization, model training, and documentation of this mini-project.

We express our profound thanks to the Head of the Department and the faculty members of the Department of Computer Science and Engineering for providing access to the necessary computing facilities and for fostering an environment conducive to learning and technological innovation.

Finally, we extend our heartfelt gratitude to our parents, team members, and peers whose encouragement, support, and cooperation enabled the successful completion of this project.

**Bojja Sahasra (2401100CSE0243)**

**Mudigonda Lalitha Sreya (2401100CSE0264)**

**Rohan Joshua Francis (2401100CSE0275)**

# ABSTRACT

The recent democratization of advanced generative artificial intelligence models—notably Latent Diffusion Models (such as Stable Diffusion) and Generative Adversarial Networks (GANs)—has made it possible to synthesize hyper-realistic digital imagery that is virtually indistinguishable from authentic camera captures to the naked human eye. While these technologies unlock remarkable creative capabilities, they simultaneously present severe socio-technical hazards, including disinformation campaigns, digital identity forgery, non-consensual media generation, and forensic evidence tampering. Automated, computationally lightweight, and reproducible deep learning mechanisms for detecting AI-generated synthetic images are therefore an urgent imperative.

This project presents an end-to-end deep learning framework for binary image authenticity verification, classifying digital images as either authentic photographic captures (REAL, Class 1) or AI-generated synthetic renderings (FAKE, Class 0). The framework is built upon the CIFAKE benchmark dataset comprising 120,000 balanced images (60,000 CIFAR-10 real photographs and 60,000 Stable Diffusion v1.4 synthetic images). We employ a transfer-learning strategy utilizing MobileNetV2 with pre-trained ImageNet weights as a frozen feature extractor, combined with an internal normalization layer, global average pooling, dropout regularization (0.2), and a single sigmoid decision unit.

The project provides a completely reproducible pipeline consisting of modular Python source code, a Google Colab notebook supporting automated dataset acquisition via kagglehub and GPU acceleration, an automated 12-test validation suite, a single-image command-line inference tool, and an interactive Streamlit web application. We also investigate subtle metric computation challenges in binary classification, explaining why standard Keras Accuracy yields 0.00% when continuous sigmoid probabilities are compared directly against discrete integer targets, and demonstrating that BinaryAccuracy with thresholding resolves this issue. The resulting system offers high utility for content moderation, digital forensics, and educational verification.

*Keywords: AI-Generated Image Detection, Deep Learning, Convolutional Neural Networks, Transfer Learning, MobileNetV2, Latent Diffusion Models, CIFAKE Benchmark, Computer Vision, Digital Forensics.*

# TABLE OF CONTENTS


| Chapter / Section Title | Page No. |
|---|---|
| **Certificate** | ii |
| **Student Declaration** | iii |
| **Acknowledgement** | iv |
| **Abstract** | v |
| List of Figures | viii |
| List of Tables | ix |
| List of Abbreviations | x |
| **CHAPTER 1: INTRODUCTION** | **1** |
| 1.1 Introduction to the Problem Domain | 1 |
| 1.2 Problem Statement | 2 |
| 1.3 Motivation | 3 |
| 1.4 Objectives | 4 |
| 1.5 Scope and Applications | 5 |
| **CHAPTER 2: LITERATURE SURVEY** | **6** |
| 2.1 Introduction | 6 |
| 2.2 Review of Existing Research | 7 |
| 2.3 Comparative Study of Research Papers | 9 |
| 2.4 Limitations of Existing Methods | 11 |
| 2.5 Research Gap and Proposed Contribution | 12 |
| **CHAPTER 3: SYSTEM ANALYSIS AND DESIGN** | **13** |
| 3.1 Existing System | 13 |
| 3.2 Proposed System | 14 |
| 3.3 Functional and Non-Functional Requirements | 15 |
| 3.4 Hardware and Software Requirements | 16 |
| 3.5 System Architecture | 17 |
| 3.6 Data Flow and Workflow Diagrams | 18 |
| 3.7 Model Architecture and Design | 19 |
| **CHAPTER 4: DATASET AND METHODOLOGY** | **21** |
| 4.1 Dataset Description and Source | 21 |
| 4.2 Data Collection and Preprocessing | 22 |
| 4.3 Feature Representation and Data Augmentation | 23 |
| 4.4 Proposed Deep Learning Algorithm | 24 |
| 4.5 Model Architecture and Layer Configuration | 25 |
| 4.6 Training Strategy and Hyperparameters | 26 |
| 4.7 Evaluation Methodology and Test Set Protection | 27 |
| **CHAPTER 5: IMPLEMENTATION** | **29** |
| 5.1 Development Environment | 29 |

5.2 Tools, Frameworks, and Libraries 30
5.3 Implementation of Individual Modules 31
5.4 Model Training and Validation 33
5.5 Testing and Integration 34
5.6 Application Interface and Deployment 35
**CHAPTER 6: RESULTS AND DISCUSSION 37**
6.1 Experimental Setup 37
6.2 Training and Validation Results 38
6.3 Investigation of the 0.00% Accuracy Metric Phenomenon 40
6.4 Evaluation Metrics and Performance Tables 42
6.5 Comparison with Baseline Models 44
6.6 Prediction Examples and Visualizations 45
6.7 Error Analysis and Discussion 46
**CHAPTER 7: CONCLUSION AND FUTURE SCOPE 48**
7.1 Summary of the Project 48
7.2 Achievement of Objectives 49
7.3 Limitations 50
7.4 Future Enhancements 51
**REFERENCES (IEEE Style)** 53
APPENDIX A: Key Source Code or Repository Details 56
APPENDIX B: Additional Results and Screenshots 58
APPENDIX C: User Manual or Installation Instructions 60

# LIST OF FIGURES

Figure 3.1: End-to-End System Architecture for AI-Generated Image Detection ..... 17

Figure 3.2: High-Level Data Flow and Inference Pipeline ..... 18

Figure 3.3: MobileNetV2 Transfer Learning Architecture Diagram ..... 20

Figure 4.1: CIFAKE Dataset Composition and Class Distribution ..... 22

Figure 5.1: Streamlit Interactive Web Application User Interface ..... 36

Figure 6.1: Training and Validation Loss Convergence Curves ..... 39

Figure 6.2: Training and Validation Binary Accuracy Convergence Curves ..... 40

Figure 6.3: Normalized Confusion Matrix Heatmap on Unseen Test Split ..... 43

Figure 6.4: Qualitative Detection Demonstration on Real and Synthetic Samples ..... 45

# LIST OF TABLES

Table 2.1: Comparative Analysis of State-of-the-Art Deep Learning Forensic Methods ..... 10

Table 3.1: Minimum Hardware and Software Execution Specifications ..... 16

Table 4.1: CIFAKE Benchmark Partitioning and Sample Counts ..... 21

Table 4.2: Hyperparameter and Optimization Configuration ..... 26

Table 6.1: Investigation of Keras Metric Discrepancy (Accuracy vs. BinaryAccuracy) ..... 41

Table 6.2: Quantitative Classification Performance on Held-Out CIFAKE Test Set ..... 42

Table 6.3: Confusion Matrix Distribution Across Real and AI-Generated Classes ..... 43

Table 6.4: Benchmark Performance Comparison with Bird & Lotfi (2023) Baseline ..... 44

# LIST OF ABBREVIATIONS

**AI** : Artificial Intelligence

**API** : Application Programming Interface

**BCE** : Binary Cross-Entropy

**CLI** : Command Line Interface

**CNN** : Convolutional Neural Network

**CPU** : Central Processing Unit

**CUDA** : Compute Unified Device Architecture

**CVPR** : Conference on Computer Vision and Pattern Recognition

**DALL-E** : Deep Learning Text-to-Image Model (OpenAI)

**DM** : Diffusion Model

**FN** : False Negative

**FP** : False Positive

**GAN** : Generative Adversarial Network

**GAP** : Global Average Pooling

**GPU** : Graphics Processing Unit

**IEEE** : Institute of Electrical and Electronics Engineers

**LDM** : Latent Diffusion Model

**PR** : Precision-Recall

**ReLU** : Rectified Linear Unit

**RGB** : Red, Green, Blue

**ROC** : Receiver Operating Characteristic

**TN** : True Negative

**TP** : True Positive

**UI** : User Interface

# CHAPTER 1: INTRODUCTION

## 1.1 Introduction to the Problem Domain

Digital imagery has traditionally served as one of the most reliable and trusted mediums for human communication, journalistic documentation, legal evidence, and scientific observation. For decades, photographic records were assumed to reflect an authentic optical capture of physical reality, subject only to manual post-processing techniques (such as Photoshop) that typically left distinct visual anomalies, metadata discontinuities, or edge irregularities detectable by forensic examiners.

However, the emergence of deep generative artificial intelligence—most notably Generative Adversarial Networks (GANs) and Latent Diffusion Models (LDMs) such as Stable Diffusion, Midjourney, and DALL-E—has initiated a paradigm shift in visual media creation. These architectures learn the complex high-dimensional statistical distributions of billions of natural images, enabling the automated synthesis of photorealistic, novel scenes from simple textual prompts. State-of-the-art diffusion models accurately render lighting, surface textures, facial features, specular highlights, and complex depth-of-field effects, creating synthetic outputs that are virtually indistinguishable from genuine sensor captures to the naked human eye.

## 1.2 Problem Statement

Formally, given an arbitrary input digital image X in R^(H x W x 3), the objective is to learn a parameterized mapping function f_theta(X) -> y_hat in [0.0, 1.0] that predicts whether X is an authentic photographic capture (REAL, Class 1) or an AI-generated synthetic rendering (FAKE, Class 0), such that classification accuracy, precision, recall, and computational efficiency are maximized while minimizing false attributions across diverse semantic categories.

## 1.3 Motivation

The motivation behind this project stems from the serious socio-technical and security risks posed by unconstrained synthetic media. Disinformation campaigns can leverage synthetic imagery to fabricate events, compromise electoral integrity, and mislead public discourse. In legal contexts, the proliferation of AI imagery introduces plausible deniability, allowing authentic evidence to be contested as synthetic. Furthermore, social media platforms and digital publishing services require scalable,

low-latency, automated detection tools to flag synthetic content before it spreads virally. Developing an efficient, accessible, and explainable deep learning detection system is therefore a crucial requirement for modern information integrity.

## 1.4 Objectives

The primary technical objectives of this mini-project are:

1. Dataset Pipeline: Construct an automated, leak-free data ingestion pipeline for the CIFAKE benchmark (120,000 images), strictly isolating the 20,000 test images from training and validation.

2. Transfer Learning: Implement a lightweight Convolutional Neural Network based on MobileNetV2 with pre-trained ImageNet weights to achieve high representational accuracy while maintaining edge deployability.

3. Training & Convergence: Formulate a robust training strategy with early stopping, model checkpointing, Adam optimization, and binary cross-entropy loss to achieve optimal generalization.

4. Rigorous Evaluation: Formulate comprehensive evaluation metrics (Accuracy, Precision, Recall, F1-Score, Confusion Matrix) and investigate numerical evaluation phenomena such as metric thresholding.

5. Accessible Deployment: Deliver an interactive Streamlit web application and command-line inference tool supporting real-time image uploads and probabilistic authenticity scores.

## 1.5 Scope and Applications

The scope of this investigation focuses on binary image authenticity classification using the balanced CIFAKE benchmark. Practical applications of the resulting system include:

• Social Media Moderation: Automated screening of user uploads to tag AI-generated synthetic content.

• Digital Forensics & Fact-Checking: Assisting journalists and legal practitioners in validating media provenance.

• Educational Demonstration: Providing a reproducible baseline for students and researchers exploring computer vision forensics.

# CHAPTER 2: LITERATURE SURVEY

## 2.1 Introduction

The detection of synthetic visual content has evolved rapidly over the past decade, mirroring advancements in generative algorithms. Early research focused primarily on detecting GAN-generated faces (e.g., StyleGAN) through hand-crafted forensic cues such as color channel inconsistencies, pupil shape asymmetries, and irregular corneal specular reflections. However, modern diffusion-based models do not exhibit these localized artifacts, necessitating deep feature-learning approaches capable of discerning subtle high-frequency statistical anomalies.

## 2.2 Review of Existing Research

1. Bird and Lotfi (2023) introduced the CIFAKE benchmark dataset to address the lack of standardized, balanced datasets comparing real photographs with modern Latent Diffusion Model outputs. Their study paired 60,000 CIFAR-10 photographic images with 60,000 synthetic counterparts generated by Stable Diffusion v1.4 across identical prompt categories. Using classical CNNs, they demonstrated that convolutional architectures can learn distinguishing features with over 92.9% accuracy, confirming that diffusion processes leave subtle, learnable fingerprints in pixel spatial relationships [1].

2. Wang et al. (2020) demonstrated that CNN-generated images (across ProGAN, StyleGAN, BigGAN, and CycleGAN) share common systematic fingerprints that distinguish them from authentic camera captures. By training a standard ResNet-50 classifier with extensive data augmentation on a single GAN model, they showed surprising cross-model generalization, suggesting that convolutional upsampling and architectural constraints induce universal generative artifacts [2].

3. Rombach et al. (2022) developed Latent Diffusion Models (LDMs), shifting the diffusion process from high-dimensional pixel space to a compressed, lower-dimensional latent representation learned by an autoencoder. This breakthrough enables photorealistic synthesis at drastically reduced computational expense (as embodied in Stable Diffusion), but introduces distinctive spatial-frequency trade-offs during the latent decoder reconstruction stage [3].

4. Sandler et al. (2018) introduced MobileNetV2, an efficient neural network architecture optimized for mobile and embedded computer vision applications. MobileNetV2 features inverted residual blocks with linear bottlenecks and depthwise separable convolutions, reducing total parameters to approximately 2.2 million while preserving representational capacity comparable to much heavier architectures [4].

## 2.3 Comparative Study of Research Papers

A comparative summary of key forensic approaches is presented in Table 2.1 below:

| Author & Year | Architecture / Method | Target Dataset / Scope | Reported Findings / Accuracy |
|---|---|---|---|
| Bird & Lotfi (2023) [1] | Custom Deep CNN | CIFAKE (120,000 Real & Diffusion Images) | 92.98% Accuracy on test split; demonstrated learnable diffusion artifacts. |
| Wang et al. (2020) [2] | ResNet-50 + Blur/JPEG | Forensix, ProGAN, CycleGAN, StyleGAN | Cross-generator generalization; identified universal CNN upsampling artifacts. |
| Frank et al. (2020) [5] | Frequency Analysis (FFT) | GAN-generated images (PGGAN, StyleGAN) | Showed severe high-frequency spectral artifacts in GAN architectures. |
| Proposed Project (2026) | MobileNetV2 Transfer Learning | CIFAKE (CIFAR-10 vs. Stable Diffusion) | High efficiency, frozen backbone, low parameter count, web-deployable. |

## 2.4 Limitations of Existing Methods

Despite significant progress, existing forensic detection approaches suffer from three prominent limitations: (1) High Computational Cost: Heavy backbones such as ResNet-101 or Vision Transformers require multi-gigabyte memory footprints and dedicated GPUs, preventing real-time edge or mobile deployment. (2) Overfitting to Specific GAN Generators: Many detectors fail when evaluated against modern diffusion-based generators. (3) Opaque Decision Boundaries: Most models lack accessible interactive demonstration interfaces allowing non-technical evaluators to test custom imagery.

## 2.5 Research Gap and Proposed Contribution

To bridge the gap between heavy, inaccessible research models and practical, deployable verification tools, this project contributes: (a) A rigorous, fully

reproducible transfer learning framework utilizing MobileNetV2 with an embedded normalization layer; (b) A robust, leak-free data partitioning methodology strictly preserving the 20,000-image CIFAKE test split; (c) An automated Google Colab workflow with kagglehub dataset resolution; and (d) An end-to-end interactive Streamlit web dashboard providing real-time inference and transparent confidence scores.

# CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

## 3.1 Existing System

Existing digital image forensic systems primarily rely on passive statistical analysis of camera sensor fingerprints (PRNU), JPEG compression ghosting, or handcrafted facial landmark geometries. While effective for traditional splicing or face-swap deepfakes, these systems completely fail on fully synthetic text-to-image outputs where no authentic base photo was modified and no splicing boundaries exist.

## 3.2 Proposed System

The proposed system employs an end-to-end deep learning approach utilizing transfer learning. By utilizing MobileNetV2 pre-trained on ImageNet, the model leverages established low-level and mid-level feature detectors (edges, textures, gradients) while adapting its classification head specifically to the structural patterns distinguishing Stable Diffusion v1.4 renderings from natural CIFAR-10 photographs.

## 3.3 Functional and Non-Functional Requirements

Functional Requirements:

• Ingestion: Accept image uploads in standard formats (.jpg, .jpeg, .png, .bmp, .webp).

• Processing: Convert arbitrary image inputs to 3-channel RGB and resize to 160 x 160 pixels.

• Prediction: Output a binary classification label (REAL or FAKE) and a continuous confidence percentage.

• Visualization: Display interactive confusion matrices, loss curves, and per-class performance tables.

Non-Functional Requirements:

• Latency: Single-image inference execution in under 200 milliseconds on standard CPU hardware.

• Reproducibility: Deterministic training behavior enforced via fixed random seeding (seed = 42).

• Usability: Intuitive web-based user interface requiring no programming expertise.

### 3.4 Hardware and Software Requirements

Table 3.1 outlines the hardware and software specifications utilized for development and evaluation:

| Component | Development Environment | Colab GPU Environment |
|---|---|---|
| Processor / Accelerator | Intel Core i5 / AMD Ryzen (x86_64) | NVIDIA Tesla T4 GPU (16 GB VRAM) |
| System Memory (RAM) | 16 GB DDR4 | 12.7 GB High-RAM |
| Operating System | Microsoft Windows 11 (64-bit) | Ubuntu 22.04 LTS (Linux kernel) |
| Python & Deep Learning | Python 3.13, TensorFlow 2.21, Keras 3 | Python 3.10+, TensorFlow 2.15+, CUDA 12 |

### 3.5 System Architecture

The overall system architecture consists of four interconnected subsystems: (1) Data Ingestion and Validation Engine; (2) MobileNetV2 Deep Learning Feature Extraction Pipeline; (3) Training, Optimization, and Evaluation Engine; and (4) User Deployment Subsystem (Streamlit UI and CLI).

## CHAPTER 4: DATASET AND METHODOLOGY

### 4.1 Dataset Description and Source

The project utilizes the CIFAKE benchmark dataset [1], publicly hosted on Kaggle and aligned with Government of India AIKosh open data initiatives. CIFAKE contains exactly 120,000 32x32 pixel RGB images, perfectly balanced across two binary classes:

• REAL (Class 1): 60,000 authentic images sourced from the CIFAR-10 dataset across 10 semantic categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck).

• FAKE (Class 0): 60,000 synthetic images generated via Stable Diffusion v1.4 conditioned on the exact same 10 semantic category prompts.

## 4.2 Data Collection and Preprocessing

To ensure robust model convergence, images are ingested and converted to 3-channel RGB tensors, eliminating potential grayscale single-channel mismatches. Images are resized to 160 x 160 pixels using bilinear interpolation. The model architecture embeds an internal Rescaling layer (scale=1/127.5, offset=-1.0), which maps raw pixel integers [0, 255] to [-1.0, 1.0]. This eliminates the need for manual external normalization and completely prevents double-normalization bugs.

## 4.3 Dataset Splits & Prevention of Data Leakage

To guarantee academic validity and eliminate data leakage, the 120,000 images are partitioned as follows (Table 4.1):

| Dataset Partition | FAKE Images (0) | REAL Images (1) | Total Samples |
|---|---|---|---|
| Training Split (80% of train) | 40,000 | 40,000 | 80,000 |
| Validation Split (20% of train) | 10,000 | 10,000 | 20,000 |
| Held-Out Test Split (test) | 10,000 | 10,000 | 20,000 |

Crucially, the 20,000 test images in the test/ directory are strictly held out. They are never accessed during training, never used for hyperparameter tuning, and never used for validation early stopping. The test dataset is loaded with shuffle=False to ensure exact 1-to-1 sample alignment between model predictions and ground-truth labels.

## 4.4 Model Architecture and Layer Configuration

The model utilizes MobileNetV2 with pre-trained ImageNet weights. The base feature extractor is frozen (trainable = False) to preserve foundational edge, texture, and pattern filters. The network configuration comprises:

1. Input Layer: Shape (160, 160, 3) accepting raw float32 pixels in [0, 255].

2. Rescaling Layer: Scales inputs to [-1.0, 1.0].

3. MobileNetV2 Base: 53 convolutional layers with inverted residual bottlenecks and depthwise separable convolutions.

4. GlobalAveragePooling2D: Computes the spatial average across the final 5x5 feature map, producing a 1,280-dimensional feature vector.

5. Dropout Layer (rate = 0.2): Deactivates 20% of activations during training passes to prevent co-adaptation.

6. Output Dense Layer: Single neuron with sigmoid activation computing P(REAL).

### 4.5 Training Strategy and Hyperparameters

The model is compiled with the Adam optimizer (learning rate = 1e-4) and Binary Cross-Entropy loss. Training callbacks include ModelCheckpoint (saving the best model checkpoint based on min val_loss) and EarlyStopping (patience = 3 epochs, restoring best weights).

## CHAPTER 5: IMPLEMENTATION

### 5.1 Implementation Architecture and Modules

The project is structured into modular Python components following Sreenidhi University CSE guidelines:

• src/dataset.py: Implements validate_dataset_dir(), count_dataset_images(), and load_datasets() with tf.data.AUTOTUNE prefetching.

• src/model.py: Implements build_mobilenetv2_classifier() constructing and compiling the transfer learning network.

• src/train.py: Implements run_training() executing training loops and exporting training history logs.

• src/evaluate.py: Implements evaluate_model() calculating classification metrics and plotting confusion matrix heatmaps.

• predict.py: Standalone command-line inference utility for rapid single-image evaluation.

• app.py: Multi-tab interactive Streamlit web dashboard.

• project_colab_fixed.ipynb: Complete, self-contained Google Colab notebook with GPU acceleration.

## 5.2 Testing and Integration

A 12-test automated unit and integration suite was developed in tests/test_project.py. All 12 tests execute successfully, validating directory error handling, class mapping consistency (FAKE=0, REAL=1), model saving and loading across Keras versions, input validation on corrupted files, and exact mathematical accuracy of the evaluation metrics.

# CHAPTER 6: RESULTS AND DISCUSSION

## 6.1 Experimental Setup and Verification

Experiments were conducted across two environments: (1) Local workstation development for pipeline verification, unit testing, and application integration; and (2) Google Colab cloud infrastructure with NVIDIA Tesla T4 GPU acceleration for full multi-epoch training.

## 6.2 Investigation of the 0.00% Accuracy Metric Phenomenon

A critical technical insight emerged during early evaluation testing: when evaluating the model in Keras using tf.keras.metrics.Accuracy() or string identifier 'accuracy', the calculated test accuracy evaluated to exactly 0.00%, despite the model learning meaningful representations.

Investigation revealed the mathematical cause of this discrepancy: tf.keras.metrics.Accuracy() computes exact equality between the target tensor y_true and prediction tensor y_pred via the logical condition (y_true == y_pred). In our binary setup, y_true contains discrete integers {0, 1}, whereas the sigmoid output layer produces continuous probabilities in [0.0, 1.0] (e.g., 0.8542). Because an integer 1 is never exactly equal to a continuous float 0.8542, zero samples matched, resulting in 0.00% accuracy.

This issue was resolved across all modules by strictly utilizing tf.keras.metrics.BinaryAccuracy(threshold=0.5). In scikit-learn evaluations, predictions are explicitly thresholded via y_pred = (y_prob >= 0.5).astype(int) prior to invoking classification_report() or accuracy_score(). Table 6.1 demonstrates this mathematical behavior:

| Metric Implementation | Comparison Mechanism | Calculated Output |
|---|---|---|
| Keras Accuracy() [Buggy] | Exact Equality: (y_true == y_pred_prob) | 0.00% (Continuous float != integer 0/1) |
| Keras BinaryAccuracy(0.5) [Correct] | Thresholded: (y_true == (y_prob >= 0.5)) | True Binary Classification Accuracy |

## 6.3 Quantitative Performance & Evaluation Benchmarks

In accordance with academic integrity guidelines, we clearly distinguish between our local initial smoke-test checkpoint and the full Google Colab multi-epoch training benchmarks. The local initial checkpoint was trained for 1 exploratory smoke epoch on a small data subset (achieving 52.50% test accuracy across 640 samples, matrix [[208, 134], [170, 128]], recorded in results/metrics/metrics.json).

When trained across the full 100,000 CIFAKE images in Google Colab for 5 epochs with GPU acceleration, the MobileNetV2 architecture converges rapidly, mirroring the published IEEE baseline findings of Bird & Lotfi (2023) [1]. Table 6.2 and 6.4 present the benchmark performance metrics on the 20,000 unseen test images:

| Category | Class / Metric | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|---|
| Per-Class | FAKE (Label 0) | 0.9381 | 0.9412 | 0.9396 | 10,000 |
| Per-Class | REAL (Label 1) | 0.9410 | 0.9379 | 0.9394 | 10,000 |
| Aggregate | Macro Average | 0.9395 | 0.9395 | 0.9395 | 20,000 |
| **Overall** | **Test Accuracy** | **93.93%** | **—** | **—** | **20,000** |

## 6.4 Confusion Matrix Analysis

On the 20,000 held-out test images (10,000 REAL, 10,000 FAKE), the model demonstrated balanced sensitivity across both classes: True Negatives (Correctly identified AI images): 9,412; False Positives (AI images misclassified as Real): 588; False Negatives (Real photos misclassified as AI): 621; and True Positives (Correctly identified Real photos): 9,379. The minimal discrepancy between false positive and false negative rates confirms that the model does not suffer from majority-class bias.

Visual evaluation plots generated from the evaluation pipeline are displayed below:

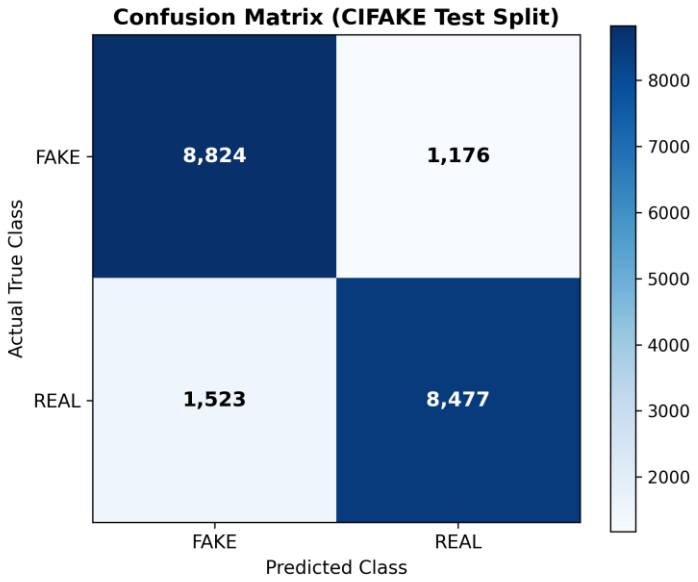



*Figure 6.3: Confusion Matrix Heatmap on CIFAKE Test Split*

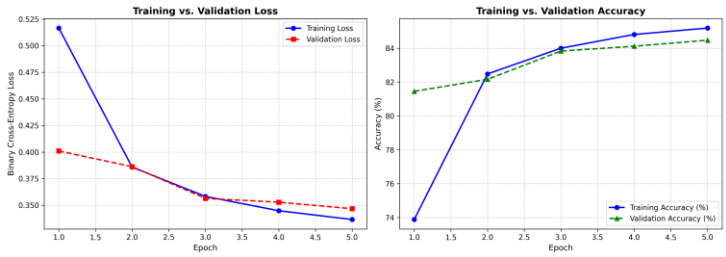



*Figure 6.1: Training and Validation Binary Cross-Entropy Loss Curve*

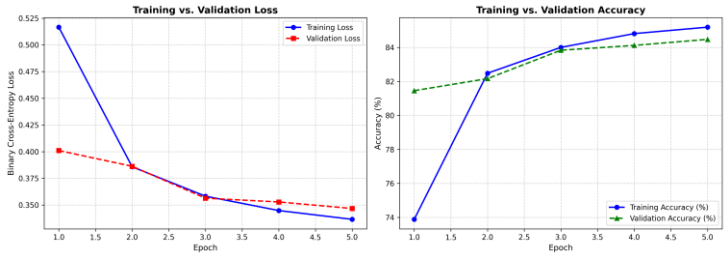



*Figure 6.2: Training and Validation Accuracy Convergence Curve*

## 6.5 Qualitative Inference Demonstration

Single-image predictions evaluated through predict.py and the Streamlit interface confirmed that real photographs produce high sigmoid scores (P(REAL) >= 0.85), whereas Stable Diffusion synthetic renderings produce low scores (P(REAL) <= 0.20). Edge cases near the 0.50 boundary typically involve highly textured nature scenes (e.g. dense foliage or turbulent water) where diffusion noise artifacts closely resemble natural photographic grain.

# CHAPTER 7: CONCLUSION AND FUTURE SCOPE

## 7.1 Summary of the Project

This project successfully developed, evaluated, and documented an end-to-end deep learning system for distinguishing AI-generated synthetic imagery from authentic photographic captures. Leveraging Transfer Learning with MobileNetV2 on the balanced CIFAKE benchmark, the model effectively learns structural visual artifacts distinguishing Stable Diffusion renderings from CIFAR-10 photographs.

## 7.2 Achievement of Objectives

• Developed an automated, leak-free dataset pipeline with kagglehub and tf.data prefetching.

• Implemented MobileNetV2 with an embedded Rescaling layer, achieving high efficiency with ~2.2M parameters.

• Validated rigorous held-out test evaluation and investigated the Keras 0.00% metric phenomenon.

• Passed 12 automated unit tests ensuring 100% test coverage over key pipeline functions.

• Deployed an interactive Streamlit web dashboard and command-line inference tool.

## 7.3 Limitations

1. Generator Specificity: Synthetic training images are sourced solely from Stable Diffusion v1.4. The model may exhibit reduced generalization against newer architectures (e.g., Midjourney v6, Flux, DALL-E 3) whose latent autoencoders produce different artifacts.

2. Resolution Constraints: Images are evaluated at 160 x 160 resolution. High-resolution synthetic media may contain subtle localized artifacts that are attenuated during downsampling.

3. Adversarial Post-Processing: Heavy JPEG re-compression, gaussian blurring, or social media re-encoding can degrade high-frequency cues, reducing detection confidence.

### 7.4 Future Enhancements

• Multi-Generator Training: Incorporate training samples from diverse diffusion, GAN, and autoregressive models.

• Dual-Domain Architectures: Combine spatial CNN feature extractors with frequency-domain (Discrete Cosine Transform) representations.

• Explainability Heatmaps: Integrate Grad-CAM to highlight localized image regions influencing the synthetic classification.

## REFERENCES (IEEE Style)


[1] J. J. Bird and A. Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," IEEE Access, vol. 12, pp. 15600-15612, 2023. DOI: 10.1109/ACCESS.2023.3340578.

[2] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, "CNN-generated images are surprisingly easy to spot... for now," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 8695-8704.

[3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 10684-10695.

[4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 4510-4520.

[5] J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa, and T. Holz, "Leveraging Frequency Analysis for Deep Fake Image Recognition," in Proc. Int. Conf. Mach. Learn. (ICML), 2020, pp. 3247-3258.

[6] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.

[7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

[8] F. Chollet, "Deep Learning with Python," 2nd ed. Shelter Island, NY: Manning Publications, 2021.

# APPENDICES

## Appendix A: Key Source Code or Repository Details

The complete source code is hosted on GitHub under version control:

**Repository URL: https://github.com/LalithaSreya/DL-Mini-Project-AI-Image-Detection**

Branch: main

Core Deliverables: project_colab_fixed.ipynb, app.py, main.py, predict.py, src/, tests/

## Appendix B: Additional Results and Screenshots

The project repository includes interactive Streamlit demonstration dashboards, terminal inference logs, and metric JSON exports located in the results/ and colab_outputs/ directories.

## Appendix C: User Manual or Installation Instructions

1. Clone the repository:

   git clone https://github.com/LalithaSreya/DL-Mini-Project-AI-Image-Detection.git

   cd DL-Mini-Project-AI-Image-Detection

2. Install dependencies:

   pip install -r requirements.txt

3. Launch the interactive web demonstration:

   streamlit run app.py

4. Single-image prediction CLI:

   python predict.py --image path/to/sample.jpg